The Software Life Cycle Revisited

CSE 362

Software Development and Professional Practice

Winter 2023

© Royal Military College of Canada

Introduction

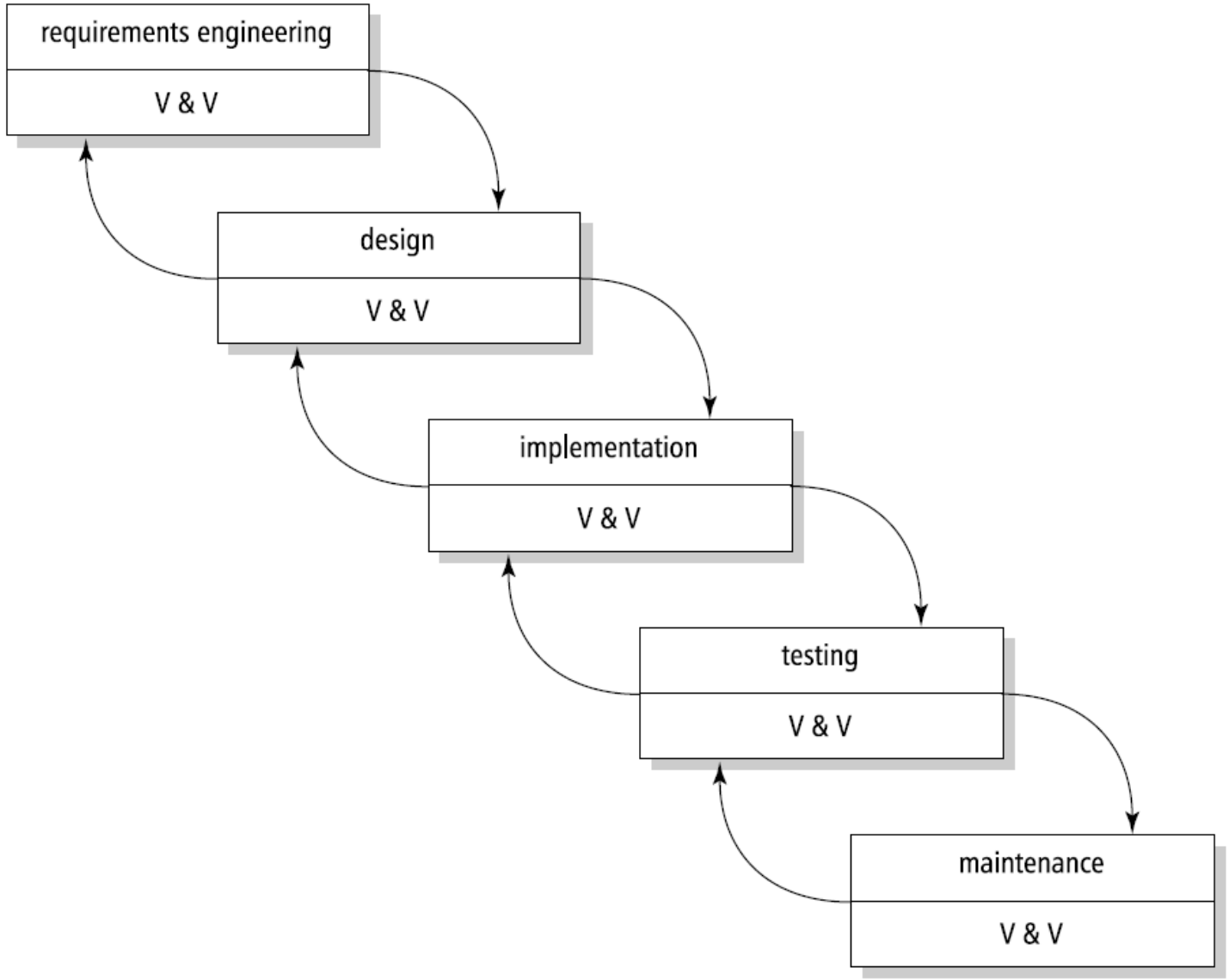
- To be able to assess progress during software development, need to use a phased approach.
  - A number of well-defined milestone events.

- Last week, we identified the SD phases:
  - requirements engineering, design, implementation, testing, maintenance.

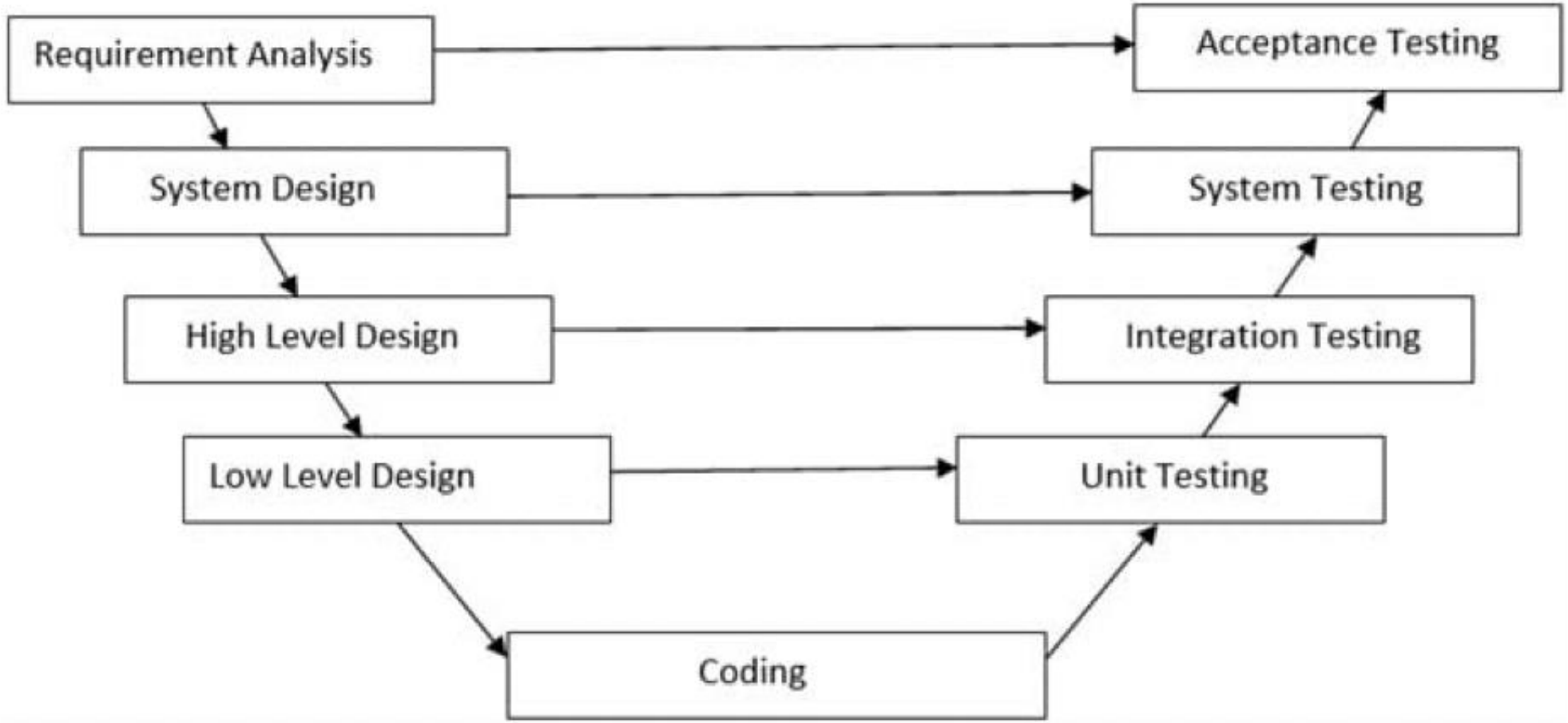
- In practice, various alternative models are used to structure the software development process.

Planning-driven
Document-driven
Heavyweight

Waterfall Model



V-shape Model



What happens in practice

In reality, the strict sequencing of phases advocated by the waterfall model is not actually obeyed.

Activity	Phase			
	Design	Coding	Integration testing	Acceptance testing
Integration testing	4.7	43.4	26.1	25.8
Coding	6.9	70.3	15.9	6.9
Design	49.2	34.1	10.3	6.4

Agile methods
Lightweight
Change-driven

Principles of Agile Software Development

- **Individuals and interactions** over processes and tools.
- **Working software** over comprehensive documentation.
- **Customer collaboration** over contract negotiation.
- **Responding to change** over following a plan.

Pure Agile Development

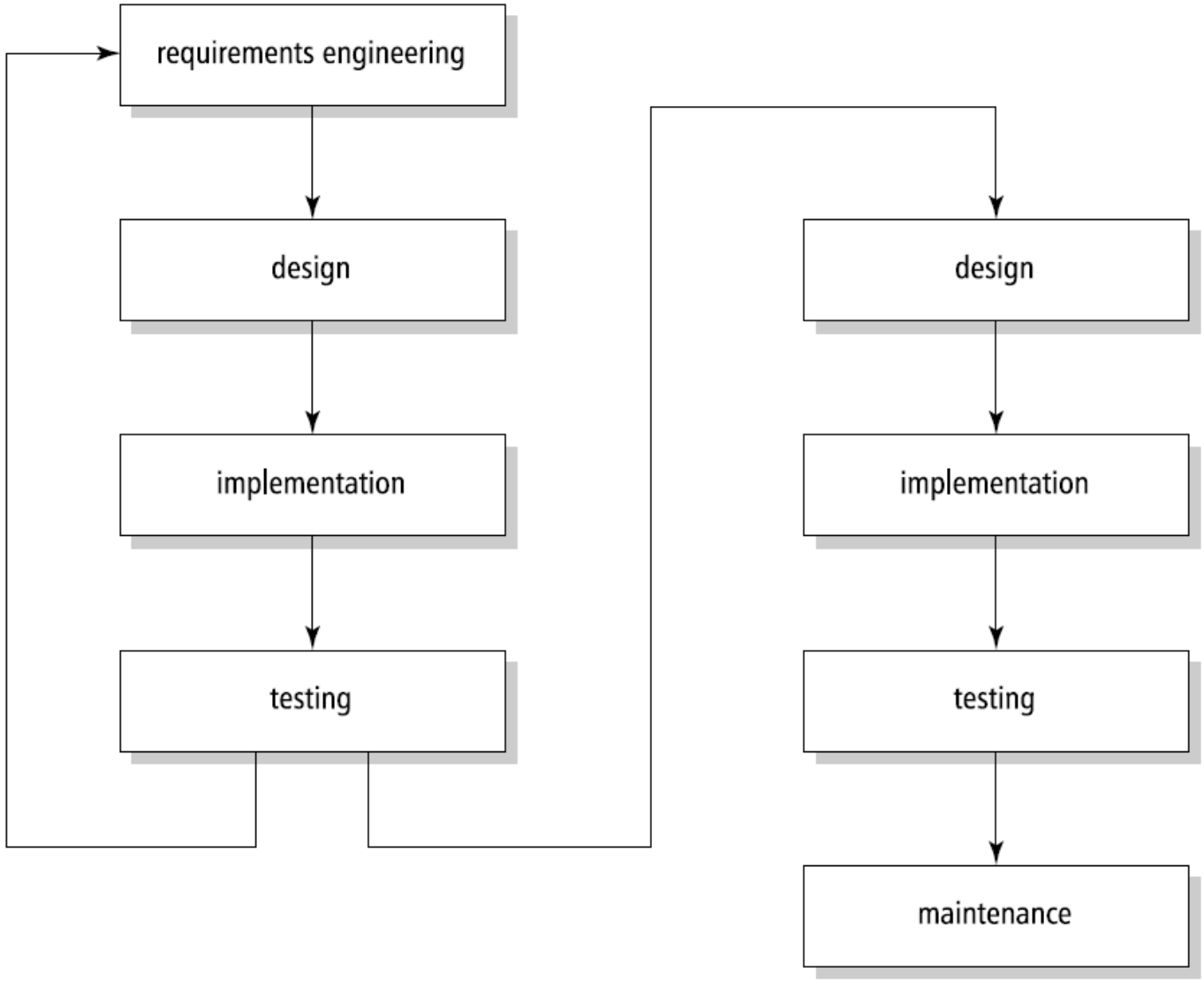
- Involve the user in every step.
- Development cycles are small and incremental.
- At the end of each cycle, the system is up and running.
- No extensive architectural or design phase up front.
- Minimal effort on documentation: it will change anyhow, why waste time?
- Team spirit and collaboration: if you have a question, don't look at documents, ask one of your pals.

Agile Development

- Some people believe that agile methods should be ‘pure’, and exhibit all of the characteristics mentioned.

- Others believe that a mixture of planning-driven and agile methods can be productive as well.
  - What could go wrong with pure Agile?

Prototyping



Prototyping: 2 types

- *Throwaway prototyping:* when the software product is not carried over from the prototyping phases to the actual production phase,

- *Evolutionary prototyping:* After a number of iterations, the user is satisfied and the last version developed is the product that goes over the production phase.

What could go well/wrong with prototyping?

- Users feel more involved in the development process and have fewer conflicts with the designers.
- Prototyping involves iterative design steps and, because of the repeated attention to the design, its quality may increase.

- The prototyping approach requires more experienced team members, if not the iterative portion becomes problematic.
- The resulting systems may be less maintainable (as a result of multiple reworks)

Incremental Development

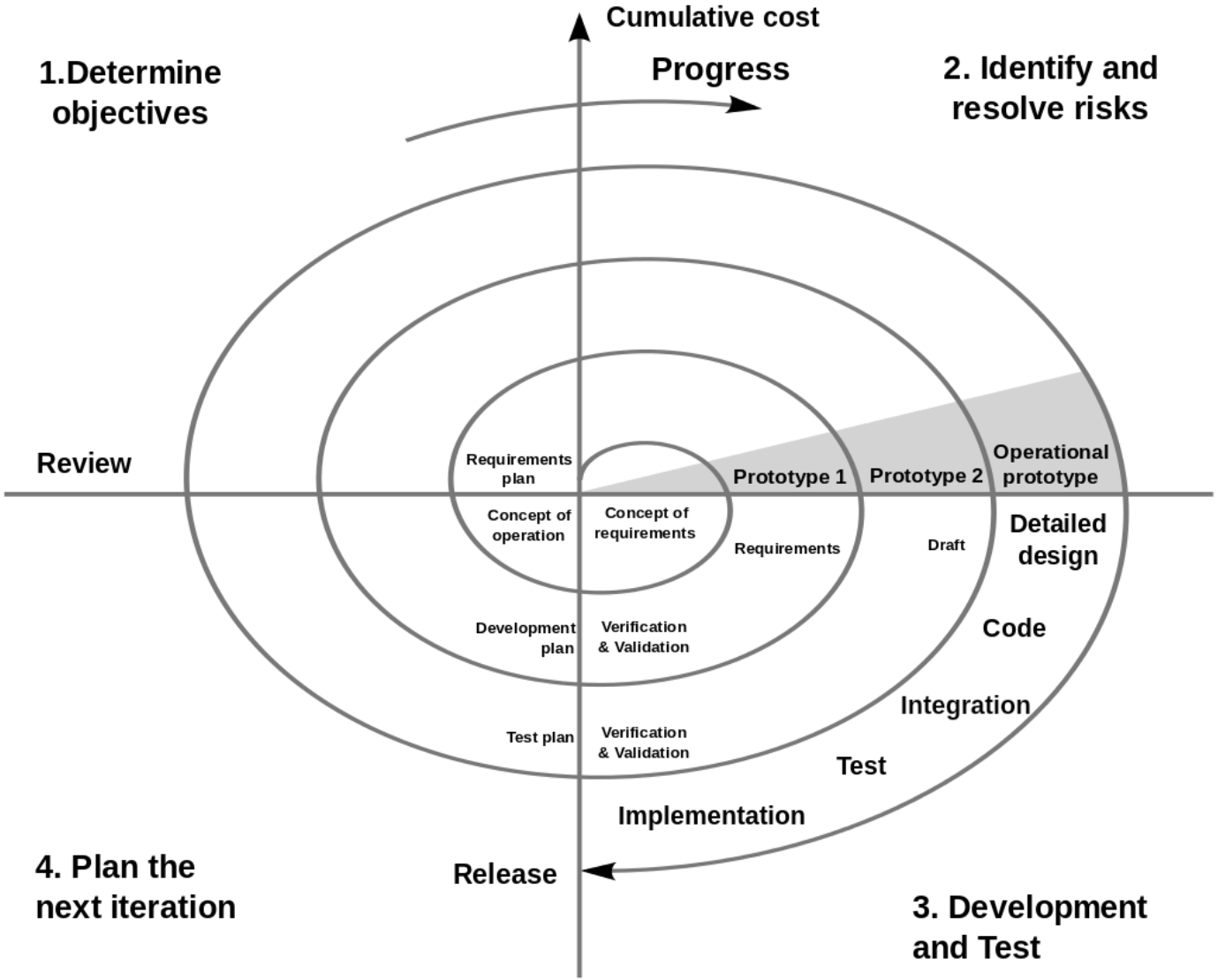
- We proceed incrementally.
- The functionality of the system is produced and delivered to the customer in *small increments*: avoids the 'Big Bang' effect.
- Instead of building software, the *software grows*.

- Help to fight the 'overfunctionality' syndrome.
- With the incremental approach, attention is first focused on the essential features.
  - Additional functionality is only included if and when it is needed

Incremental Development

- A risk-driven software development process model.
  - identify the sub-problem which has the highest associated risk;
  - find a solution for that problem;
- Adopt elements of one or more process models, such as incremental, waterfall, or prototyping.

Spiral Model



Rapid Application Development (RAD)

- has a lot in common with other iterative development process models.
- emphasizes user involvement, prototyping, reuse, the use of automated tools, and small teams
- What's different: it employs the notion of a *time box*, a fixed time frame within which activities are done.

- The RAD life cycle consists of four phases:
  - requirements planning,
  - user design,
  - construction,
  - cutover

Extreme Programming (XP)

- A pure agile method.

- XP is based on a number of best practices that have been known for long. *XP takes these practices to extreme*

- For example, it is known that code reading by your pals, such as is done in code inspections is a very effective test method.
- In XP, one does this all the time: two programmers work together behind one computer screen all the time: pair programming.

Practices of XP

XP practice	Description
The Planning Game	The scope of the next release is quickly determined. When necessary, the plan is updated
Small releases	First realize a simple system, then release next versions in short cycles
Metaphor	Use of a simple metaphor for the whole system
Simple design	Make sure the design is as simple as possible at any point in time. Remove complexity as soon as possible
Testing	Programmers continuously write unit tests, customers write acceptance tests
Refactoring	Restructure the system without changing its behaviour, to improve quality
Pair programming	All code is written by two programmers at one machine
Collective ownership	Anyone can change any code, anywhere, at any time
Continuous integration	The system is integrated and built many times a day
40-hour week	As a rule, work 40 hours a week. Working overtime should be the exception
On-site customer	Have a real user on the team, full-time
Coding standards	Establish coding standards to ease communication

Planning in XP

- Planning meetings are very short, and involve the immediate functionality to be delivered to the customer.
- Planning involves both the customer and the technical people.
- The customer describes desirable features of the system in stories, on index cards.
- The technical people estimate how long it takes to implement a story, decide on the order in which stories within one release will be implemented